

Vector Search

Do you really need another
database?

26th September 2026 | Pune Meetup

Safe Harbour Statement

The views, opinions, and conclusions presented in these slides are solely my own and do not reflect the official stance, policies, or perspectives of my employer or any affiliated organization. Any statements made are based on my personal experiences and research and should not be interpreted as official guidance or endorsement.

About us



Pranav Mehta

MTS-4 @ Nutanix

- Passionate About Distributed Systems
- Excited about:
 - Clickhouse, Vector Databases, P2P, Linux
- Ping: [linkedin.com/in/pranavmehta94/](https://www.linkedin.com/in/pranavmehta94/)
- Tech talks: github.com/pranavmehta94/tech-talks
- Medium: medium.com/@pranavmehta94



Kailash Kejriwal

MTS-1 @ Nutanix

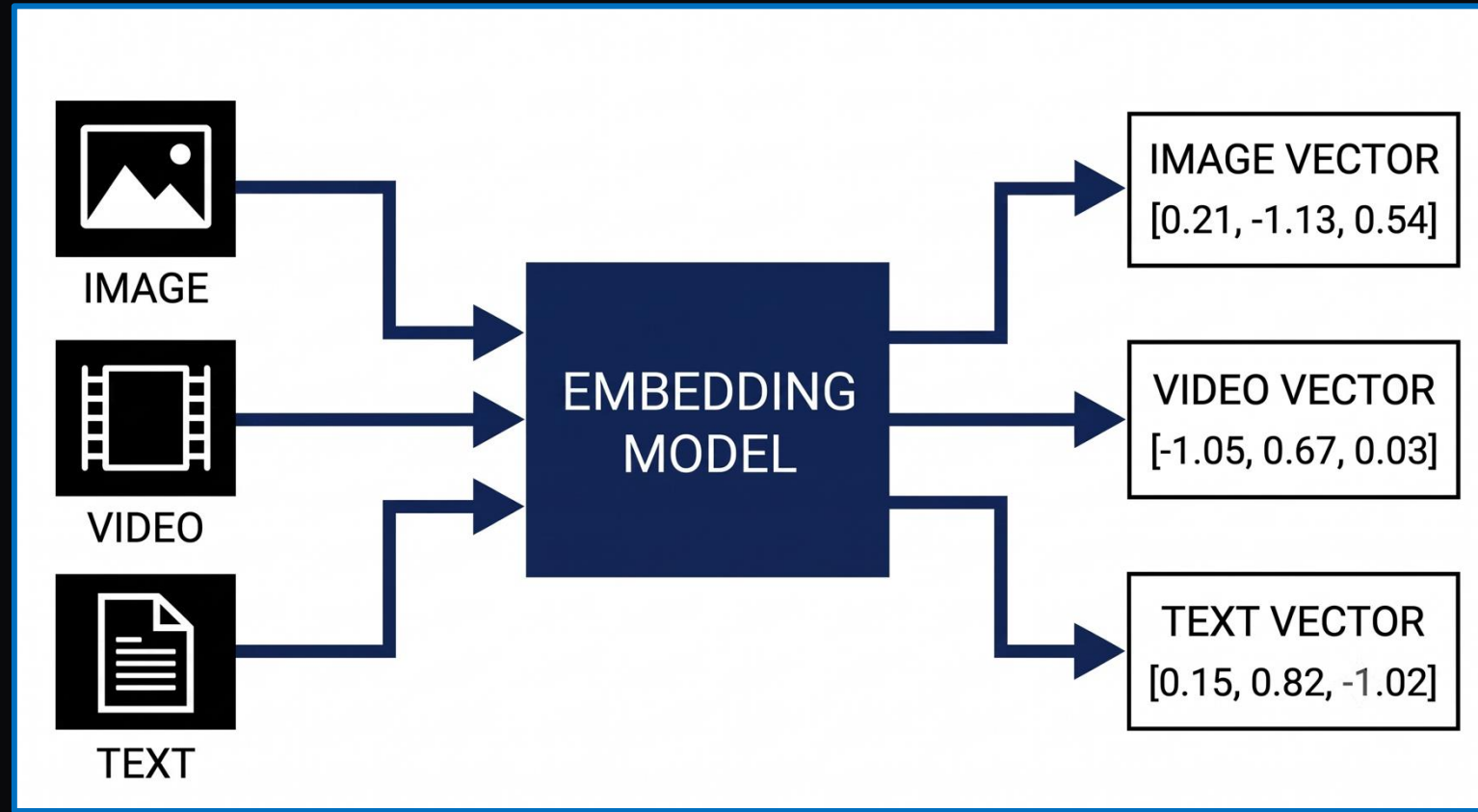
- Building & Scaling Distributed Systems
- Excited about:
 - Vector Search, Databases, Qdrant
- Ping: [linkedin.com/in/kailash360/](https://www.linkedin.com/in/kailash360/)
- Medium: medium.com/@kailash360

What is Vector Search?

Understanding the fundamental building blocks

What are Vectors?

1. In the field of artificial intelligence, vectors are mathematical representation of unstructured data
2. Unstructured data includes text, images, videos, audio
3. Represented as an array of floating-point numbers
4. Generated using an embedding model



Types of Vector

Dense Vectors



Dense vector is an array with non-zero values



The length of the array can typically vary between 512-4096



Examples include text/audio/video embeddings



Vector Search implies dense vector search

Sparse Vectors

A sparse vector is a array where the vast majority of the values are zero

The length of the array can be as high as 100,000

This format is most commonly used in traditional keyword search (like TF-IDF or BM25) to map documents to a fixed dictionary of words.

Dimensionality of Vectors

A "vector" is an array of floating-point numbers

The "dimension" is simply the total count of numbers in that array.

What is Vector Search?

- Vector search is the process of finding semantically “similar” items over unstructured data
- The search returns a bunch of relevant items e.g. top 10

Imagine a government officer has access to **1 million government documents** and wants to find:

“Find amendments to Rajasthan’s 2019 water policy.”

With **keyword search**, the system looks for documents containing words like “**Rajasthan,**” “**2019,**” “**water,**” “**policy,**” and “**amendment.**”

But imagine the actual document says:

*“The State Government hereby **revises certain provisions** of the State Water Policy, 2019, relating to allocation of water resources.”*

It **never uses the word “amendment.”**

A **keyword** search may miss it.

With **vector search**, the query and documents are converted into vectors that capture their meaning. The system understands that:

“amendment to a policy” ≈ “revision of provisions of a policy”

So the document can be retrieved even though the exact word “**amendment**” doesn't appear.

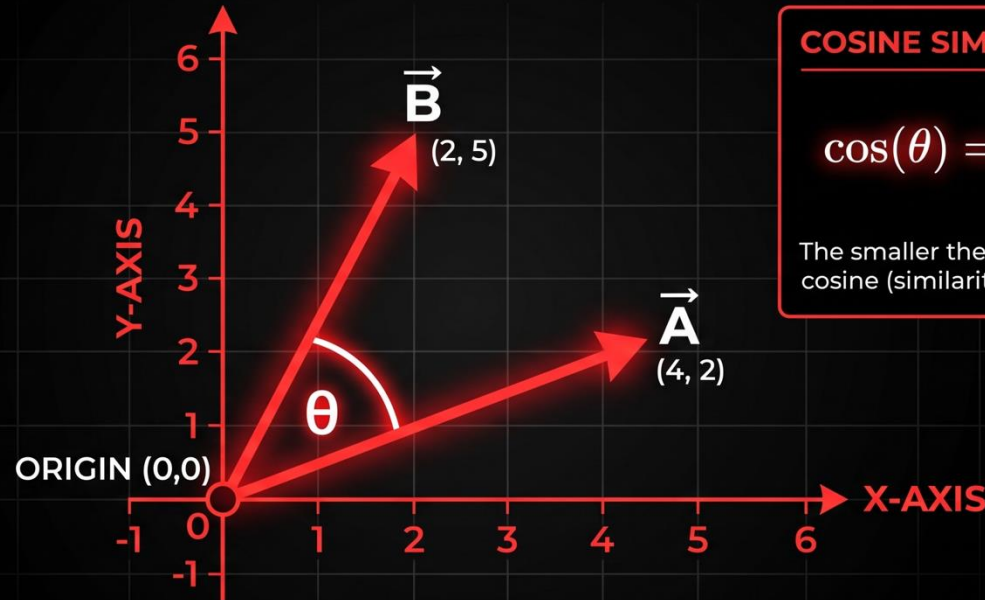
Comparing Vectors

“Similarity” of vectors can be determined using Cosine similarity or Euclidean distance between two vectors

Embedding models ensures the query vector and corpus vectors are placed close enough in a vector space based on their semantic meaning

Cosine Similarity

ANGLE BETWEEN TWO 2D VECTORS & COSINE SIMILARITY



COSINE SIMILARITY (sim)

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

The smaller the θ , the greater the cosine (similarity), approaching 1.

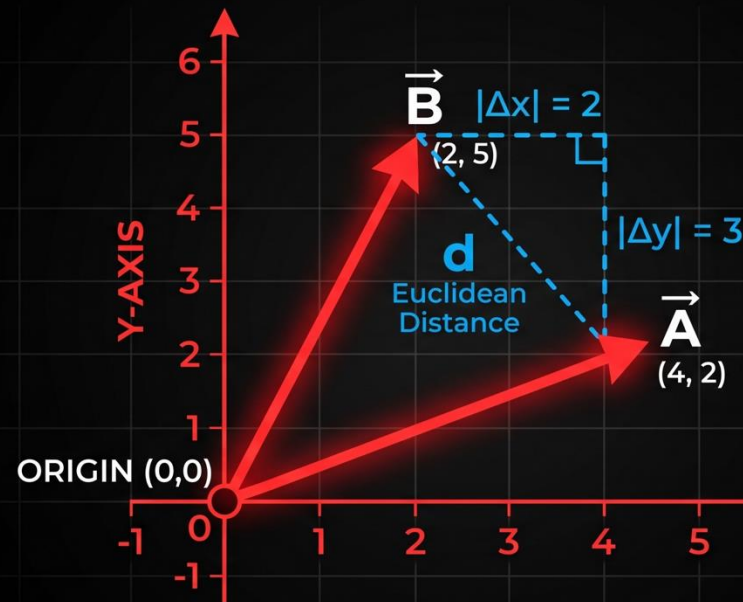
$\vec{A} = [4, 2]$ (Red Vector)

$\vec{B} = [2, 5]$ (Red Vector)

θ = Angle Between $\vec{A}$ and $\vec{B}$

Euclidean Distance

EUCLIDEAN DISTANCE BETWEEN TWO 2D VECTORS



EUCLIDEAN DISTANCE (d)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d = \sqrt{(4 - 2)^2 + (2 - 5)^2}$$

$$= \sqrt{(2)^2 + (-3)^2}$$

$$= \sqrt{4 + 9} = \sqrt{13}$$

$$d \approx 3.61 \text{ units}$$

The straight-line distance between the endpoints of the vectors.

$$\vec{A} = [4, 2] \text{ (Red Vector)}$$

$$\vec{B} = [2, 5] \text{ (Red Vector)}$$

Approximate Nearest Neighbour

1. Finding nearest neighbour over large dataset
e.g. 1 billion document is time consuming
2. To speed up searches, find approximate
nearest neighbours instead of true nearest
neighbours
3. Vector Indexes like HNSW solve ANN search
over vector data
4. ANN implies you will likely miss relevant items
5. Search quality of ANN search is measured with
a metric called recall@k

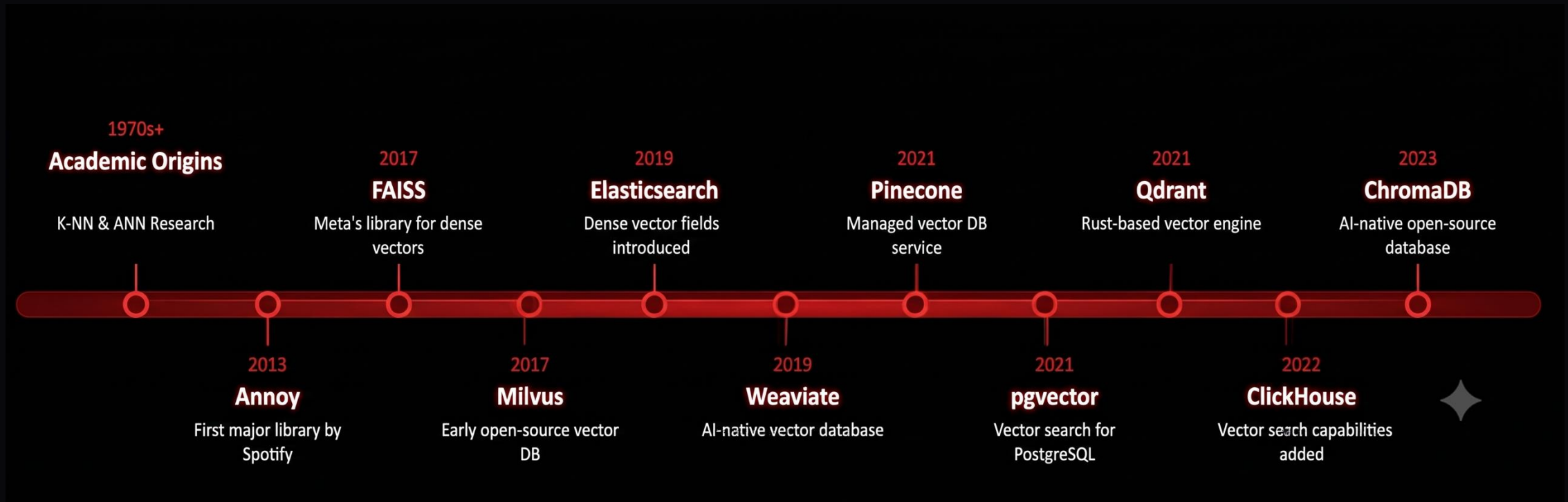
What does Vector Search Solve (Use cases)

Vector search handles unstructured data in ways traditional relational databases simply cannot, enabling modern AI applications.

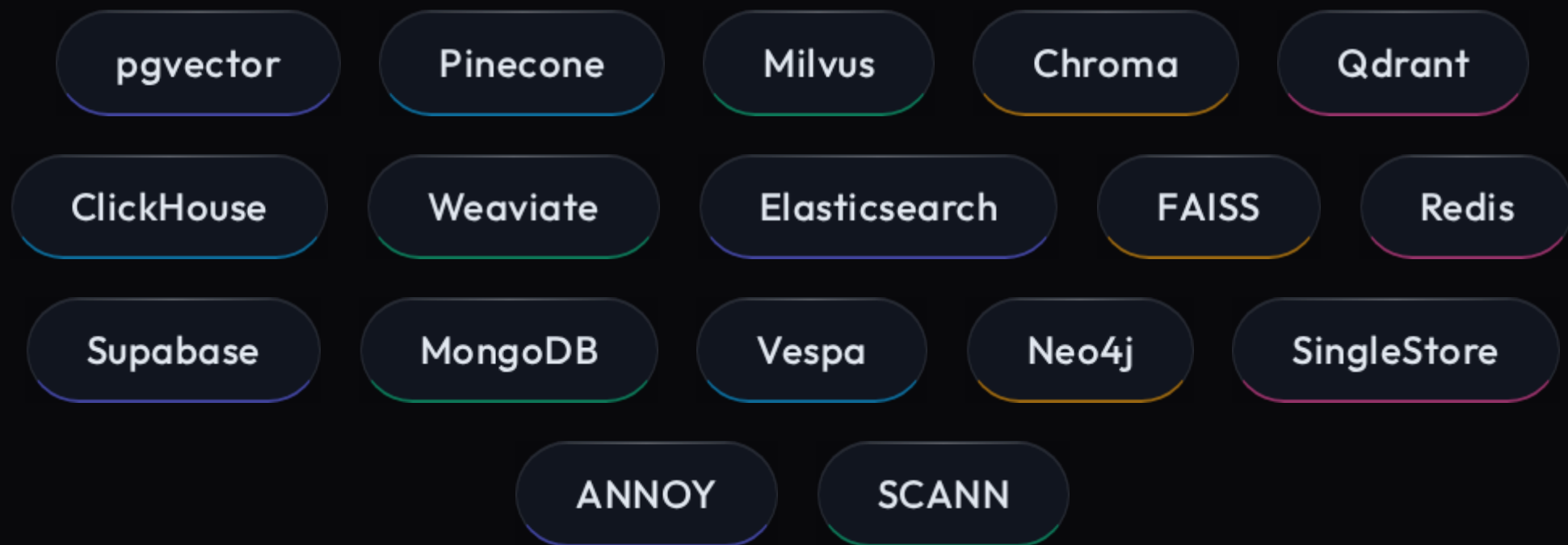
Use cases include

1. Retrieval Augment Generation in LLMs and search systems like Perplexity AI
2. Recommendation Engines in Spotify, Dailymotion
3. Image search via caption in Google Photos

The evolution of Vector Search



The Vector Database Ecosystem



The Database Dilemma

How to choose the right database?



Scale & Dataset Size

Are you storing 100K vectors or 1 Billion vectors?



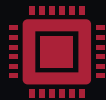
Search with Filters

Do you need to filter heavily before searching?



Vector + Keyword (Hybrid) Search

Do you also need to search specific keywords in your text?



Resource Efficiency

How much resource can you invest?



Operational Complexity

How much infrastructure can your team manage?

How about the
existing
databases?

PostgreSQL (pgVector)

The pragmatic default.

Pros

- ✓ Zero new infrastructure.
- ✓ ACID compliance.
- ✓ SQL based

Cons

- Filtering before search can be slower.
- Lack of native hybrid search
- Restrictive architecture for very high scale scenarios

ClickHouse

The analytical powerhouse.

Pros

- ✓ Incredible speed for analytics and filtering.
- ✓ Re-use existing data infrastructure

Cons

- Not a dedicated vector search engine
- Lack of native hybrid search
- Pre-filtering problem
- Relies on external vector search library

What's so special
about
Specialized
Vector Databases?

“When you cross 50M-100M vectors,
or need complex filtering at low latency,
this is where **purpose-built engines shine.**”

Milvus

The Scale Monster (Cloud-Native Architecture)

Strengths

- ✓ Highly distributed
- ✓ Separates compute, storage, and metadata
- ✓ Massive ingestion throughput

Perfect Fit

- Billions of vectors
- Enterprise deployments needing high concurrency and multiple indexing strategies

Qdrant

The Filtering Champion

Strengths

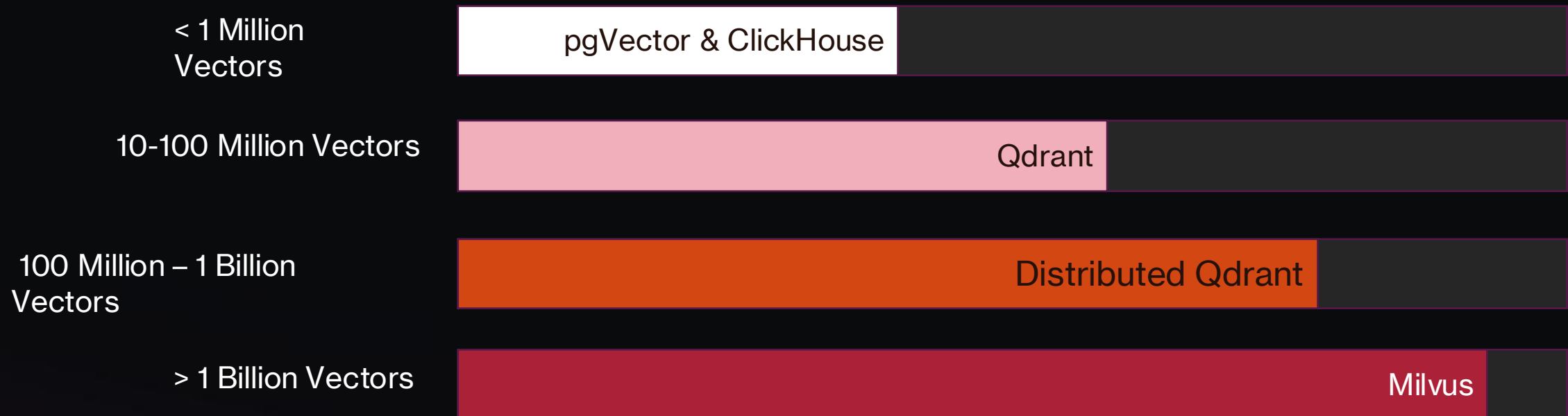
- ✓ Extreme memory efficiency.
- ✓ Excels at "filtered search"
- ✓ Easy to maintain and operate

Perfect Fit

- Easily handles 10M - 100M vector dataset on a single node
- Low operational overhead for multi-node setups

The “Rule of Thumb” Framework

Based on data scale, we can choose a database.



Thank You

Questions?

Pranav Mehta
Kailash Kejriwal